Ecole Nationale Supérieure Polytechnique de Douala

National Higher Polytechnic School of Douala

Département de Génie Informatique et Télécommunications

Intitulé

THEORIE DU LANGAGE ET COMPILATION

Projet

Conception et Implémentation d'un Compilateur pour le Langage LGLo

Cycle Ingénieur – Génie Logiciel 04 – Alternance

Année académique 2025/2026

Sous la supervision de :

- M. MANDA ABEGA, *Enseignant principal*

Groupe 02 :

- NJAI-LOBE Daniella, *Chef de groupe*
- KAMDJO Félix Junior
- KAMFO KAMFO IDRISS
- LISSOM Kenzo
- MBADZO TAYO Richenelle
- KOMBOU Danielle

Sommaire

1 Introduction 3
2 Philosophie de conception 4
2.1 Tableau des mots clés originaux 4
2.2 Types Primitifs et littéraux 5
3 Grammaire formelle G = (VN, VT, AXI, P) 5
3.1 Vocabulaire non-terminal VN 5
3.2 Vocabulaire terminal VT 6
4 Règles de production BNF 6
4.1 Programme (Axiome) 6
4.2 Instruction 6
4.3 Bloc 7
4.4 Types 7
4.5 Déclarations 7
4.6 Affectation (conservée + étendue) 7
4.7 Expressions arithmétiques 7
4.8 Expressions booléennes 8
4.9 Lecture et écriture 8
4.10 Structure conditionnelle 9
4.11 Structures répétitives 9
4.12 Définition de fonctions (functio) 9
4.13 Définition de procédures (actio) 9
4.14 Paramètres et arguments 9
5 Exemple de programme LGLo 11
5.1 Déclarations et lecture 11
5.2 Fonction récursive 11
5.3 Procédures et boucles 12
6 Notes de conception et justifications 12
6.1 Classe de la grammaire 12

6.2 Distinction functio / actio 12
6.3 La boucle iter 13
6.4 imm vs mut 13
6.5 Le terminateur # 13

1 Introduction

La **théorie des langages et la compilation** constituent un pilier fondamental de l'informatique, permettant de comprendre la transformation d'un langage de programmation en instructions exécutables. Dans ce cadre, le présent projet consiste à **concevoir et implémenter un compilateur pour le langage LGLo**, à partir d'une grammaire initiale minimaliste limitée à une simple instruction d'affectation.

L'objectif a été d'**étendre ce langage** en y intégrant les principales constructions d'un langage général : déclarations, entrées-sorties, structures conditionnelles et répétitives, ainsi que fonctions et procédures. La **liberté accordée** dans le choix du lexique et de la syntaxe a permis de proposer un langage cohérent et original, **nommé LGLo (Langage de Génération Logique).**

Ce document constitue la **documentation technique du projet**. Il présente la grammaire formelle complète du langage, les règles BNF organisées selon les différentes structures, ainsi que les spécifications et justifications des choix de conception. Enfin, des exemples de programmes illustrent concrètement l'utilisation du langage et valident sa cohérence.

2 Philosophie de conception

LGLo emprunte la **rigueur typographique de C/C++** (accolades, point-virgule terminateur remplacé par #, opérateur **<--** issu de la grammaire originale), **la sécurité de Rust** (déclaration explicite de mutabilité avec **mut**), et **la clarté expressive de plusieurs langages**. Une **couche d'originalité** est introduite à travers un vocabulaire issu du latin technique — pragmatique, mémorisable, et résolument distinct des langages populaires.

2.1 Tableau des mots clés originaux

Tableau 1 Mots originaux du langage

Rôle	Mot-clé LGLo	Inspiration / Justification
Déclaration variable	**mut**	Rust — mutable, honnête sur la mutabilité
Déclaration constante	**imm**	Latin immutabilis — court et précis
Lecture	**captum**	Latin capere (capturer) — lire = capturer une entrée
Écriture	**emittum**	Latin emittere (émettre) — écrire = émettre vers la sortie
Fonction	**functio**	Latin direct — plus précis que fun ou func
Procédure	**actio**	Latin actio (action) — agit sans retourner de valeur
Retour de fonction	**reddere**	Latin reddere (rendre) — sémantiquement juste
Condition	**quia**	Latin quia (si/car) — distinct de tout if connu
Sinon	**aliter**	Latin aliter (autrement) — élégant, distinct de else
Boucle while	**dum**	Latin dum (tant que) — compact et expressif
Boucle for	**iter**	Latin iter (chemin) — for = itérer un chemin
Fin instruction	**#**	Conservé de la grammaire originale
Affectation	**<--**	Conservé de la grammaire originale

2.2 Types Primitifs et littéraux

Fidèle à la philosophie latine du langage, les types primitifs et les valeurs littérales de LGLo sont également nommés à partir du latin. Ce choix garantit une cohérence lexicale totale : un développeur lisant du code LGLo retrouve immédiatement la même empreinte stylistique dans les mots-clés de contrôle et dans les annotations de type. Le tableau ci-dessous recense ces types et littéraux ainsi que leurs équivalents dans les langages conventionnels.

Tableau 2 Types primitifs et littéraux

Rôle / Concept	Mot-clé LGLo	Équivalent courant	Inspiration / Justification
Entier	**`integer`**	`int / int32`	Latin integer (entier, non touché)
Nombre réel	**`reales`**	`float / double`	Latin reales (réel, relatif à la réalité mesurable)
Chaîne de caractères	**`catena`**	`string / char*`	Latin catena (chaîne, lien)
Booléen	**`bina`**	`bool / boolean`	Latin bina (par deux — vrai ou faux)
Vide / sans retour	**`nullum`**	`void / null`	Latin nullum (nul, inexistant)
Vrai	**`verum`**	`true`	Latin verum (vrai, certain)
Faux	**`falsum`**	`false`	Latin falsum (faux, trompeur)

3 Grammaire formelle G = (VN, VT, AXI, P)

3.1 Vocabulaire non-terminal VN

Tableau 3 Vocabulaire non terminal

VN — Non-terminaux
{ AXI, PROG, DECL, INST, AFFECT, LECTURE, ECRITURE, COND, BOUCLE_DUM, BOUCLE_ITER, DEF_FUNCTIO, DEF_ACTIO, APPEL,

EXPR, EXPR_ARITH, TERME, FACTEUR,
EXPR_BOOL, COMP, PARAMS, LISTE_PARAMS,
ARGS, LISTE_ARGS, TYPE, BLOC }

3.2 Vocabulaire terminal VT

Tableau 4 Vocabulaire des terminaux

VT — Terminaux
{ identi, reales, catena, bina, <--, #, ', mut, imm, captum, emittum, functio, actio, reddere, quia, aliter, dum, iter, in, de, jusqua, +, -, *, /, %, <, >, <=, >=, ==, !=, &&, ||, !, (,), {, }, ',', :, ->, integer, reales, catena, bina, nullum }

4 Règles de production BNF

4.1 Programme (Axiome)

- **AXI** → **INST AXI**
  | **INST**

4.2 Instruction

- **INST** → **AFFECT #** *// terminateur d'instruction*
  | **DECL #**
  | **LECTURE #**
  | **ECRITURE #**
  | **COND** *// pas de # : délimitée par bloc*
  | **BOUCLE_DUM**
  | **BOUCLE_ITER**
  | **DEF_FUNCTIO**
  | **DEF_ACTIO**

```
        |  APPEL  #
        |  reddere EXPR  #    // dans un bloc functio
```

4.3 Bloc

- **BLOC** → **{ AXI }** // délimiteurs obligatoires

4.4 Types

- **TYPE** → **integer**
  - | **reales**
  - | **catena**
  - | **bina**
  - | **nullum** // void – pour les actio sans retour

4.5 Déclarations

- **DECL** → **mut TYPE** identi // variable non initialisée
  - | **mut TYPE** identi <-- **EXPR** // variable initialisée
  - | **imm TYPE** identi <-- **EXPR** // constante : valeur obligatoire

4.6 Affectation (conservée + étendue)

- **AFFECT** → identi <-- nombre // origine : affectation d'un nombre
  - | identi <-- ' identi ' // origine : copie d'identifiant
  - | identi <-- **EXPR** // extension : toute expression
  - | identi <-- **APPEL** // extension : résultat de functio

4.7 Expressions arithmétiques

- **EXPR** → **EXPR_ARITH**
  - | **EXPR_BOOL**
  - | chaine

- **EXPR_ARITH** → **EXPR_ARITH + TERME**
  - | **EXPR_ARITH - TERME**
  - | **TERME**

- **TERME** → **TERME** * **FACTEUR**
  | **TERME** / **FACTEUR**
  | **TERME** % **FACTEUR**
  | **FACTEUR**

- **FACTEUR** → nombre
  | identi
  | (**EXPR_ARITH**)
  | - **FACTEUR** *// négation unaire*
  | **APPEL**

4.8 Expressions booléennes

- **EXPR_BOOL** → **EXPR_BOOL** && **COMP**
  | **EXPR_BOOL** || **COMP**
  | ! **COMP**
  | **COMP**

- **COMP** → **EXPR_ARITH** < **EXPR_ARITH**
  | **EXPR_ARITH** > **EXPR_ARITH**
  | **EXPR_ARITH** <= **EXPR_ARITH**
  | **EXPR_ARITH** >= **EXPR_ARITH**
  | **EXPR_ARITH** == **EXPR_ARITH**
  | **EXPR_ARITH** != **EXPR_ARITH**
  | booleen
  | (**EXPR_BOOL**)

4.9 Lecture et écriture

- **LECTURE** → **captum**(identi) *// lit depuis l'entrée standard*

- **ECRITURE** → **emittum**(**EXPR**)
  | **emittum**(**LISTE_ARGS**) *// écriture multi-arguments*

4.10 Structure conditionnelle

- **COND** → **quia**(**EXPR_BOOL**) **BLOC**
  | **quia**(**EXPR_BOOL**) **BLOC** **aliter** **BLOC**
  | **quia**(**EXPR_BOOL**) **BLOC** **aliter** **COND** *// chaîne de conditions*

4.11 Structures répétitives

- **BOUCLE_DUM** → **dum**(**EXPR_BOOL**) **BLOC** *// tant que – analogue while*

- **BOUCLE_ITER** → **iter**(identi **in** **EXPR_ARITH** **de** **EXPR_ARITH** **jusqua** **EXPR_ARITH**) *// in=init, de=pas, jusqua=borne exclue*
  | **BOUCLE_ITER** **BLOC**

4.12 Définition de fonctions (functio)

- **DEF_FUNCTIO** → **functio** **TYPE** identi(**PARAMS**) **BLOC** *// retourne TYPE via reddere*

4.13 Définition de procédures (actio)

- **DEF_ACTIO** → **actio** identi(**PARAMS**) **BLOC** *// aucun retour – néant implicite*

4.14 Paramètres et arguments

- **PARAMS** → **LISTE_PARAMS**
  | *(liste vide)*

- **LISTE_PARAMS** → **TYPE** identi
  | **LISTE_PARAMS**, **TYPE** identi

- **APPEL** → identi(**ARGS**)

- **ARGS** → **LISTE_ARGS**
  | *(liste vide)*

- **LISTE_ARGS → EXPR**
  | LISTE_ARGS, EXPR

5 Exemple de programme LGLo

Le programme suivant illustre l'ensemble des constructions du langage : déclarations, constante, lecture, conditionnelle, boucles, fonction récursive et procédure.

5.1 Déclarations et lecture

Tableau 5 Déclarations et lecture

Code LGLo

```
// Constante : imm exige une valeur immédiate
imm entier MAX <-- 100 #

// Variable mutable non initialisée
mut entier compteur #

// Lecture depuis l'entrée standard
captum(compteur) #
```

5.2 Fonction récursive

Tableau 6 Fonction récursive

Code LGLo

```
// functio : retourne un entier
functio entier factorielle(integer n) {
  quia(n <= 1) {
    reddere 1 #
  }
  reddere n * factorielle(n - 1) #
}
```

5.3 Procédures et boucles

Tableau 7 Procédures et boucles

Code LGLo

```
// actio : procedure sans retour
actio afficherTable(integer base, integer limite) {
  mut entier i <-- 1 #
  dum(i <= limite) {
    emittum(base * i) #
    i <-- i + 1 #
  }
}

// iter (for) : de = pas, jusqua = borne exclue
iter(k in 0 de 2 jusqua 10) {
  emittum(factorielle(k)) #
}

// Appel de procedure
afficherTable(7, 12) #
```

6 Notes de conception et justifications

6.1 Classe de la grammaire

La grammaire LGLo est construite dans l'esprit des grammaires LL(1) : chaque règle est déterminée par un token de décision unique (**mut**, **imm**, **captum**, **quia**, etc.). Cela simplifie directement l'écriture du fichier Bison en minimisant les conflits shift/reduce et en rendant l'analyseur prédictif.

6.2 Distinction functio / actio

La séparation explicite entre **functio** (fonction qui calcule et retourne une valeur via **reddere**) et **actio** (procédure qui agit sans retour) reflète une distinction sémantique fondamentale présente en Pascal mais absente de C/C++. Elle rend le code plus lisible et oblige le programmeur à réfléchir à la sémantique de chaque sous-programme.

6.3 La boucle iter

La syntaxe iter(i in 0 de 2 jusqua 10) se lit naturellement : "itère i, en partant de 0, par pas de 2, jusqu'à 10 exclu". Le paramètre **de** (le pas) est obligatoire, contrairement à Python où il est optionnel. Cela élimine l'ambiguïté dans l'analyse lexicale et syntaxique.

6.4 imm vs mut

Inspiré de Rust, mais simplifié : en LGLo, **imm** exige une valeur à la déclaration. Il est syntaxiquement impossible de déclarer une constante sans valeur. Le compilateur pourra vérifier cette contrainte directement à l'analyse syntaxique, sans phase sémantique dédiée.

6.5 Le terminateur

Le terminateur d'instruction # issu de la grammaire originale est conservé. Il constitue un marqueur identitaire fort pour LGLo, différenciant le langage du point-virgule ; de C/C++ et du saut de ligne de Python. Il simplifie aussi la gestion d'erreurs dans Bison : une production manquante est immédiatement détectable.